

**UNIVERSIDAD PERUANA LOS ANDES
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS Y COMPUTACIÓN**



CURSO: Base de Datos II

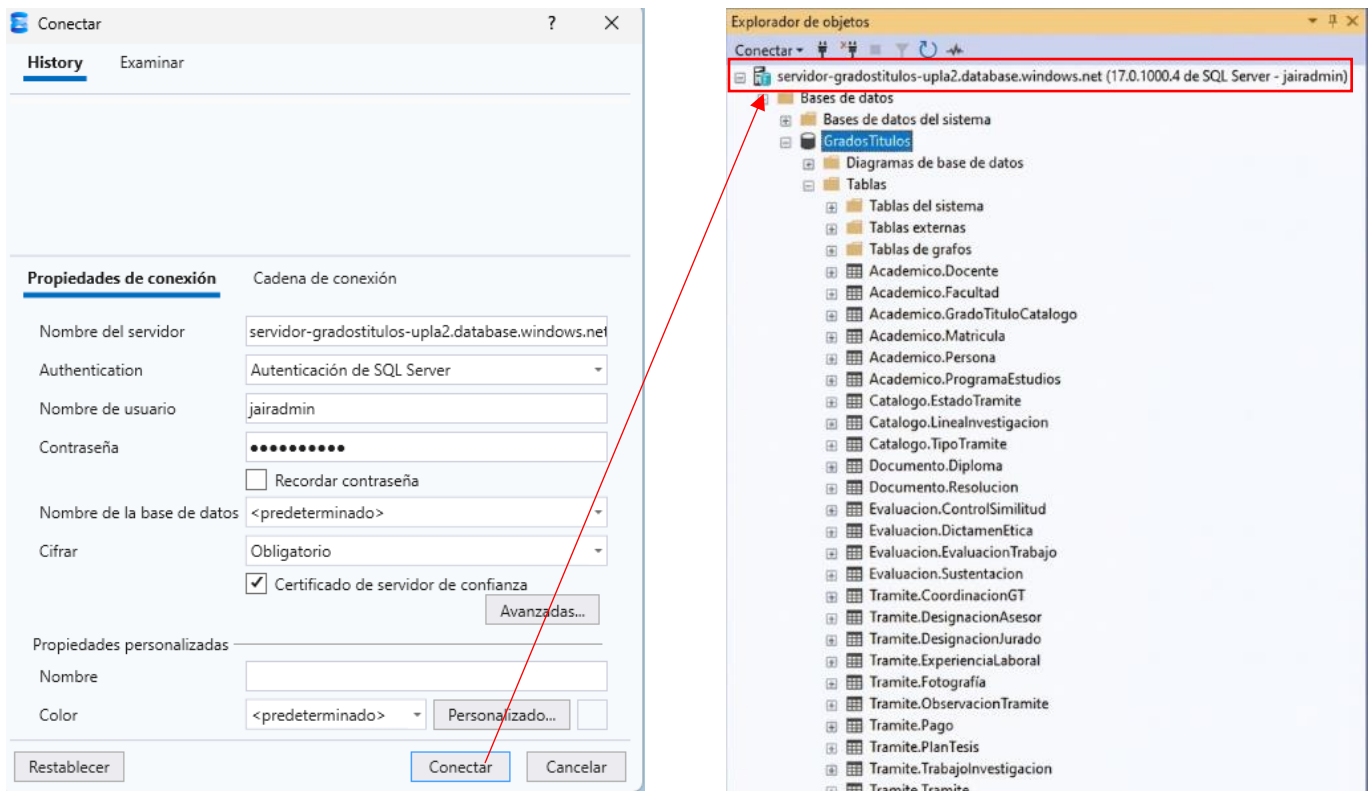
DOCENTE: Ing. Fernandez Bejarano Raul Enrique

ESTUDIANTE: Nuñez Michel Jair

CICLO: V

SECCION: B1

Conectar Azure SQL en SQL Server Management Studio 22 y desarrollo de todos los enunciados :



TEMA 1. Autenticación SQL y Windows: diferencias y prácticas seguras

Proyecto 1: Implementación de acceso mixto para personal académico y administrativo Enunciado

La Universidad requiere que los trabajadores internos accedan mediante autenticación **Windows**, mientras que los consultores externos utilicen autenticación **SQL Server**. Diseñe e implemente una estrategia de autenticación para la base de datos **GradosTitulos** que permita administrar ambos tipos de acceso.

```
USE master;
GO

-- Crear un Login de Windows para el personal interno
CREATE LOGIN [DESKTOP-01\PersonalAcademico]
FROM WINDOWS;
GO

-- Crear un Login de SQL Server para consultores externos
CREATE LOGIN ConsultorExterno
WITH PASSWORD = 'Consultor123*';
GO

USE GradosTitulos;
GO

-- Crear el usuario para el Login de Windows
CREATE USER [PersonalAcademico]
FOR LOGIN [DESKTOP-01\PersonalAcademico];
GO

-- Crear el usuario para el Login SQL
CREATE USER ConsultorExterno
FOR LOGIN ConsultorExterno;
GO

-- Asignar permisos de lectura y escritura al personal interno
ALTER ROLE db_datareader
ADD MEMBER [PersonalAcademico];
GO

ALTER ROLE db_datawriter
ADD MEMBER [PersonalAcademico];
GO

-- Asignar solo permisos de lectura al consultor externo
ALTER ROLE db_datareader
ADD MEMBER ConsultorExterno;
GO

-- Verificar los usuarios creados
SELECT
    dp.name AS Usuario,
    dp.type_desc AS TipoUsuario
FROM sys.database_principals dp
WHERE dp.name IN ('PersonalAcademico', 'ConsultorExterno');
GO
```

Justificación técnica de la solución aplicada

Se implementó una estrategia de autenticación mixta utilizando Windows Authentication para el personal interno y SQL Server Authentication para los consultores externos, permitiendo administrar ambos tipos de acceso de forma segura.

Explicación de las buenas prácticas utilizadas en el proyecto

Se asignaron únicamente los permisos necesarios a cada usuario y se verificó la creación de los accesos para garantizar una administración segura de la base de datos.

Proyecto 2: Gestión de accesos para miembros del Jurado de Tesis

Enunciado

Los miembros del jurado necesitan acceder únicamente a información de tesis y sustentaciones. Configure accesos diferenciados utilizando autenticación Windows y SQL Server.

Script de la solución en T-SQL

```
USE master;
GO

-- Crear un Login de Windows para un miembro del jurado
CREATE LOGIN [DESKTOP-01\JuradoWindows]
FROM WINDOWS;
GO

-- Crear un Login de SQL Server para un jurado externo
CREATE LOGIN JuradoSQL
WITH PASSWORD = 'Jurado123*';
GO

USE GradosTitulos;
GO

-- Crear los usuarios en la base de datos
CREATE USER JuradoWindows
FOR LOGIN [DESKTOP-01\JuradoWindows];
GO

CREATE USER JuradoSQL
FOR LOGIN JuradoSQL;
GO

-- Conceder permisos de lectura únicamente sobre las tablas de tesis y sustentaciones
GRANT SELECT
ON Academico.Tesis
TO JuradoWindows;
GO

GRANT SELECT
ON Academico.Sustentacion
TO JuradoWindows;
GO

GRANT SELECT
ON Academico.Tesis
TO JuradoSQL;
GO

GRANT SELECT
ON Academico.Sustentacion
TO JuradoSQL;
GO

-- Verificar los permisos otorgados
SELECT
    dp.name AS Usuario,
    pe.permission_name AS Permiso,
    pe.state_desc AS Estado
FROM sys.database_permissions pe
INNER JOIN sys.database_principals dp
ON pe.grantee_principal_id = dp.principal_id
WHERE dp.name IN ('JuradoWindows', 'JuradoSQL');
GO
```

Justificación técnica de la solución aplicada

Se configuraron accesos mediante autenticación Windows y SQL Server, otorgando únicamente permisos de lectura sobre la información de tesis y sustentaciones para proteger el resto de los datos institucionales.

Explicación de las buenas prácticas utilizadas en el proyecto

Se aplicó el principio de mínimo privilegio concediendo solo permisos SELECT sobre los objetos requeridos y verificando posteriormente que los permisos fueran asignados correctamente.

Proyecto 3: Gestión de accesos para miembros del Jurado de Tesis

Enunciado

La Oficina de Tecnologías de Información requiere aplicar políticas estrictas para las cuentas SQL Server que administran los trámites de grados y títulos.

Script de la solución en T-SQL

```
USE master;
GO

-- Crear el Login SQL Server para el administrador de trámites
CREATE LOGIN AdministradorTramites
WITH PASSWORD = 'AdminTramites123!',
CHECK_POLICY = ON,
CHECK_EXPIRATION = ON;
GO

USE GradosTitulos;
GO

-- Crear el usuario asociado al Login
CREATE USER AdministradorTramites
FOR LOGIN AdministradorTramites;
GO

-- Asignar permisos para administrar la información de trámites
ALTER ROLE db_datareader
ADD MEMBER AdministradorTramites;
GO

ALTER ROLE db_datawriter
ADD MEMBER AdministradorTramites;
GO

-- Verificar el Login creado
SELECT
    name AS Login,
    type_desc AS TipoLogin,
    is_policy_checked AS PoliticaContrasena,
    is_expiration_checked AS ExpiracionContrasena
FROM sys.sql_logins
WHERE name = 'AdministradorTramites';
GO
```

Justificación técnica de la solución aplicada

Se creó una cuenta de autenticación SQL Server con políticas de contraseña y expiración habilitadas, incrementando la seguridad de las cuentas encargadas de administrar los trámites de grados y títulos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se activaron las opciones CHECK_POLICY y CHECK_EXPIRATION para cumplir las políticas de seguridad de SQL Server. Además, se asignaron únicamente los permisos necesarios y se verificó la configuración del login mediante las vistas del sistema.

TEMA 2. Cuentas de servicio y configuración del servidor

Proyecto 1: Configuración segura de servicios SQL Server

Enunciado

La base de datos GradosTitulos será utilizada en producción. Configure cuentas de servicio específicas para SQL Server y SQL Server Agent.

Script de la solución en T-SQL

```
USE master;
GO

-- Consultar las cuentas de servicio configuradas
SELECT
    servicename AS Servicio,
    startup_type_desc AS TipoInicio,
    service_account AS CuentaServicio,
    status_desc AS Estado
FROM sys.dm_server_services;
GO
```

Justificación técnica de la solución aplicada

Se utilizan cuentas de servicio independientes para SQL Server y SQL Server Agent, mejorando la seguridad y facilitando la administración de los servicios del servidor.

Explicación de las buenas prácticas utilizadas en el proyecto

Se recomienda utilizar cuentas dedicadas con los permisos mínimos necesarios y verificar periódicamente su estado mediante las vistas de administración dinámica del sistema.

Proyecto 2: Monitoreo de servicios críticos del servidor

Enunciado

Implemente consultas administrativas que permitan verificar el estado de las instancias **SQL Server** relacionadas con **GradosTitulos**.

Script de la solución en T-SQL

```
USE master;
GO

-- Consultar el estado de los servicios de SQL Server
SELECT
    servicename AS Servicio,
    startup_type_desc AS TipoInicio,
    status_desc AS Estado,
    service_account AS CuentaServicio,
    last_startup_time AS UltimoInicio
FROM sys.dm_server_services;
GO

-- Consultar información de la instancia de SQL Server
SELECT
    @@SERVERNAME AS NombreServidor,
    @@SERVICENAME AS NombreInstancia,
    @@VERSION AS VersionSQLServer;
GO
```

Justificación técnica de la solución aplicada

Las consultas permiten verificar el estado de los servicios e identificar información importante de la instancia de SQL Server, facilitando el monitoreo y la detección temprana de posibles incidencias.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron vistas de administración dinámica para consultar el estado de los servicios sin afectar el rendimiento del servidor y verificar que la instancia se encuentre operativa.

Proyecto 3: Configuración de protocolos de red seguros

Enunciado

Configure la instancia **SQL Server** para permitir únicamente conexiones seguras a la base de datos **GradosTitulos**.

Script de la solución en T-SQL

```
USE master;
GO

-- Verificar si las conexiones cifradas están habilitadas
SELECT
    session_id,
    encrypt_option AS ConexionCifrada,
    protocol_type AS Protocolo,
    auth_scheme AS Autenticacion,
    client_net_address AS DireccionCliente
FROM sys.dm_exec_connections
WHERE session_id = @@SPID;
GO
```

Justificación técnica de la solución aplicada

La configuración de protocolos seguros mediante **SQL Server Configuration Manager** garantiza que las conexiones a la base de datos se realicen utilizando cifrado, protegiendo la información transmitida entre los clientes y el servidor.

Explicación de las buenas prácticas utilizadas en el proyecto

Se recomienda habilitar únicamente los protocolos necesarios, utilizar conexiones cifradas mediante SSL/TLS y verificar posteriormente el estado de las conexiones utilizando las vistas de administración dinámica de SQL Server.

TEMA 3. Creación de roles fijos y personalizados

Proyecto 1: Roles para el proceso de grados y títulos

Enunciado

Diseñe roles personalizados para gestionar el acceso a los esquemas:

- Catalogo
- Academico
- Tramite
- Evaluacion
- Documento

Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- Crear los roles personalizados
CREATE ROLE RolCatalogo;
CREATE ROLE RolAcademico;
CREATE ROLE RolTramite;
CREATE ROLE RolEvaluacion;
CREATE ROLE RolDocumento;
GO

-- Asignar permisos sobre cada esquema
GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::Catalogo
TO RolCatalogo;
GO

GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::Academico
TO RolAcademico;
GO

GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::Tramite
TO RolTramite;
GO

GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::Evaluacion
TO RolEvaluacion;
GO

GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::Documento
TO RolDocumento;
GO

-- Verificar los roles creados
SELECT
    name AS Rol,
    type_desc AS Tipo
FROM sys.database_principals
WHERE type = 'R'
AND name IN
(
    'RolCatalogo',
    'RolAcademico',
    'RolTramite',
    'RolEvaluacion',
    'RolDocumento'
);
GO

```

Justificación técnica de la solución aplicada

Se crearon roles personalizados para cada esquema de la base de datos, facilitando la administración de permisos y permitiendo asignar accesos de manera organizada según las funciones de cada área.

Explicación de las buenas prácticas utilizadas en el proyecto

Se implementó una separación de permisos por esquemas, simplificando la administración de la seguridad y favoreciendo el principio de mínimo privilegio mediante roles específicos para cada módulo del sistema.

Proyecto 2: Implementación de roles para evaluadores de tesis

Enunciado

Los evaluadores deben consultar tesis y registrar observaciones sin modificar información académica.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Crear el rol para evaluadores
CREATE ROLE RolEvaluadorTesis;
GO

-- Permitir consultar la información de tesis
GRANT SELECT
ON Academico.Tesis
TO RolEvaluadorTesis;
GO

-- Permitir consultar la información de sustentaciones
GRANT SELECT
ON Academico.Sustentacion
TO RolEvaluadorTesis;
GO

-- Permitir registrar observaciones
GRANT INSERT
ON Evaluacion.Observacion
TO RolEvaluadorTesis;
GO

-- Denegar modificaciones sobre la información académica
DENY UPDATE
ON SCHEMA::Academico
TO RolEvaluadorTesis;
GO

DENY DELETE
ON SCHEMA::Academico
TO RolEvaluadorTesis;
GO

-- Verificar los permisos del rol
SELECT
    dp.name AS Rol,
    pe.permission_name AS Permiso,
    pe.state_desc AS Estado
FROM sys.database_permissions pe
INNER JOIN sys.database_principals dp
ON pe.grantee_principal_id = dp.principal_id
WHERE dp.name = 'RolEvaluadorTesis';
GO
```

Justificación técnica de la solución aplicada

Se creó un rol específico para los evaluadores de tesis, permitiéndoles consultar la información necesaria y registrar observaciones sin otorgar permisos para modificar los datos académicos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se aplicó el principio de mínimo privilegio mediante el uso de **GRANT** y **DENY**, limitando las acciones del rol únicamente a las funciones que requiere dentro del sistema.

Proyecto 3: Administración de roles jerárquicos

Enunciado

Configure roles que representen los niveles:

- Operador
- Supervisor
- Administrador

para el sistema de grados y títulos.

Script de la solución en T-SQL

```
USE GradosTítulos;
GO

-- Crear los roles jerárquicos
CREATE ROLE RolOperador;
CREATE ROLE RolSupervisor;
CREATE ROLE RolAdministrador;
GO

-- Permisos para el Rol Operador
GRANT SELECT, INSERT
ON SCHEMA::Tramite
TO RolOperador;
GO

-- Permisos para el Rol Supervisor
GRANT SELECT, INSERT, UPDATE
ON SCHEMA::Tramite
TO RolSupervisor;
GO

GRANT SELECT
ON SCHEMA::Evaluacion
TO RolSupervisor;
GO

-- Permisos para el Rol Administrador
GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::Catalogo
TO RolAdministrador;
GO

GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::Academico
TO RolAdministrador;
GO

GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::Tramite
TO RolAdministrador;
GO

GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::Evaluacion
TO RolAdministrador;
GO

GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::Documento
TO RolAdministrador;
GO

-- Verificar los roles creados
SELECT
    name AS Rol,
    type_desc AS Tipo
FROM sys.database_principals
WHERE name IN
(
    'RolOperador',
    'RolSupervisor',
    'RolAdministrador'
);
GO
```

Justificación técnica de la solución aplicada

Se implementó una estructura jerárquica de roles para distribuir los permisos según las responsabilidades de cada nivel, mejorando el control de acceso y la administración de la seguridad en la base de datos.

Explicación de las buenas prácticas utilizadas en el proyecto

Los permisos fueron asignados de forma progresiva, otorgando únicamente las operaciones necesarias para cada rol y aplicando el principio de mínimo privilegio para fortalecer la seguridad del sistema.

TEMA 4. Control de acceso mediante GRANT, DENY y REVOKE

Proyecto 1: Roles para el proceso de grados y títulos

Enunciado

Los operadores pueden consultar datos de estudiantes, pero no modificarlos.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Crear el rol para los operadores
CREATE ROLE RolOperadores;
GO

-- Conceder permiso de lectura sobre la tabla de estudiantes
GRANT SELECT
ON Academico.Persona
TO RolOperadores;
GO

-- Denegar permisos de modificación
DENY INSERT
ON Academico.Persona
TO RolOperadores;
GO

DENY UPDATE
ON Academico.Persona
TO RolOperadores;
GO

DENY DELETE
ON Academico.Persona
TO RolOperadores;
GO

-- Verificar los permisos del rol
SELECT
    dp.name AS Rol,
    pe.permission_name AS Permiso,
    pe.state_desc AS Estado
FROM sys.database_permissions pe
INNER JOIN sys.database_principals dp
ON pe.grantee_principal_id = dp.principal_id
WHERE dp.name = 'RolOperadores';
GO
```

Justificación técnica de la solución aplicada

Se otorgó únicamente el permiso **SELECT** sobre la información de estudiantes y se denegaron las operaciones de inserción, modificación y eliminación para proteger la integridad de los datos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se aplicó el principio de mínimo privilegio utilizando **GRANT** y **DENY**, permitiendo únicamente las acciones necesarias para el rol de operador y restringiendo cualquier modificación sobre la información académica.

Proyecto 2: Restricción de acceso a resoluciones y diplomas

Enunciado

Solo la **Oficina de Grados y Títulos** puede modificar documentos oficiales.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Crear el rol para la Oficina de Grados y Títulos
CREATE ROLE RolGradosTitulos;
GO

-- Conceder permisos completos sobre el esquema Documento
GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::Documento
TO RolGradosTitulos;
GO

-- Crear el rol para otros usuarios
CREATE ROLE RolUsuarios;
GO

-- Permitir únicamente la lectura de los documentos
GRANT SELECT
ON SCHEMA::Documento
TO RolUsuarios;
GO

-- Denegar la modificación de los documentos oficiales
DENY INSERT
ON SCHEMA::Documento
TO RolUsuarios;
GO

DENY UPDATE
ON SCHEMA::Documento
TO RolUsuarios;
GO

DENY DELETE
ON SCHEMA::Documento
TO RolUsuarios;
GO

-- Verificar los permisos asignados
SELECT
    dp.name AS Rol,
    pe.permission_name AS Permiso,
    pe.state_desc AS Estado
FROM sys.database_permissions pe
INNER JOIN sys.database_principals dp
ON pe.grantee_principal_id = dp.principal_id
WHERE dp.name IN ('RolGradosTitulos', 'RolUsuarios');
GO
```

Justificación técnica de la solución aplicada

Se asignaron permisos de modificación únicamente al rol **RolGradosTitulos**, mientras que el resto de usuarios conserva únicamente permisos de consulta sobre los documentos oficiales.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó una combinación de **GRANT** y **DENY** para proteger los documentos institucionales, garantizando que solo el personal autorizado pueda realizar modificaciones y aplicando el principio de mínimo privilegio.

Proyecto 3: Revocación de permisos temporales

Enunciado

Un auditor externo requiere acceso temporal durante una semana.

Script de la solución en T-SQL

```
USE master;
GO

-- Crear el Login para el auditor externo
CREATE LOGIN AuditorExterno
WITH PASSWORD = 'Auditor123*';
GO

USE GradosTitulos;
GO

-- Crear el usuario en la base de datos
CREATE USER AuditorExterno
FOR LOGIN AuditorExterno;
GO

-- Conceder permiso temporal de lectura
GRANT SELECT
ON SCHEMA::Academico
TO AuditorExterno;
GO

-- Revocar el permiso cuando finalice la auditoría
REVOKE SELECT
ON SCHEMA::Academico
FROM AuditorExterno;
GO

-- Verificar los permisos del usuario
SELECT
    dp.name AS Usuario,
    pe.permission_name AS Permiso,
    pe.state_desc AS Estado
FROM sys.database_permissions pe
INNER JOIN sys.database_principals dp
ON pe.grantee_principal_id = dp.principal_id
WHERE dp.name = 'AuditorExterno';
GO
```

Justificación técnica de la solución aplicada

Se concedió un permiso temporal de lectura al auditor externo y posteriormente se revocó el acceso mediante **REVOKE**, garantizando que el permiso solo esté disponible durante el periodo autorizado.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó **GRANT** para habilitar el acceso temporal y **REVOKE** para retirarlo una vez concluida la auditoría, reduciendo riesgos de accesos innecesarios y fortaleciendo la seguridad de la base de datos.

TEMA 5. Cifrado y protección de datos (TDE y Always Encrypted)

Proyecto 1: Implementación de TDE para GradosTitulos

Enunciado

Proteja los archivos físicos de la base de datos GradosTitulos mediante Transparent Data Encryption (TDE).

Script de la solución en T-SQL

```
USE master;
GO

-- Crear la clave maestra
CREATE MASTER KEY
ENCRYPTION BY PASSWORD = 'MasterKey123!';
GO

-- Crear el certificado para TDE
CREATE CERTIFICATE CertificadoGradosTitulos
WITH SUBJECT = 'Certificado para Transparent Data Encryption';
GO

USE GradosTitulos;
GO

-- Crear la clave de cifrado de la base de datos
CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_256
ENCRYPTION BY SERVER CERTIFICATE CertificadoGradosTitulos;
GO

-- Habilitar Transparent Data Encryption
ALTER DATABASE GradosTitulos
SET ENCRYPTION ON;
GO

-- Verificar el estado del cifrado
SELECT
    DB_NAME(database_id) AS BaseDatos,
    encryption_state AS EstadoCifrado
FROM sys.dm_database_encryption_keys
WHERE database_id = DB_ID('GradosTitulos');
GO
```

Justificación técnica de la solución aplicada

Se implementó **Transparent Data Encryption (TDE)** para cifrar los archivos físicos de la base de datos, protegiendo la información almacenada frente al acceso no autorizado en caso de pérdida o robo de los archivos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó el algoritmo **AES_256**, recomendado por SQL Server para el cifrado de bases de datos, y posteriormente se verificó que el proceso de cifrado estuviera habilitado correctamente mediante las vistas del sistema.

Proyecto 2: Protección de datos personales del tesista

Enunciado

Implemente **Always Encrypted** para proteger los siguientes datos de los estudiantes:

- DNI
- Correo electrónico
- Teléfono

Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- Crear una Clave Maestra de Columna
CREATE COLUMN MASTER KEY CMK_GradosTitulos
WITH
(
    KEY_STORE_PROVIDER_NAME = N'MSSQL_CERTIFICATE_STORE',
    KEY_PATH = N'CurrentUser/My/GradosTitulosCMK'
);
GO

-- Crear una Clave de Cifrado de Columna
CREATE COLUMN ENCRYPTION KEY CEK_GradosTitulos
WITH VALUES
(
    COLUMN_MASTER_KEY = CMK_GradosTitulos,
    ALGORITHM = 'RSA_OAEP',
    ENCRYPTED_VALUE = 0x01
);
GO

-- Cifrar la columna DNI
ALTER TABLE Academico.Persona
ALTER COLUMN DNI VARCHAR(8)
COLLATE Latin1_General_BIN2
ENCRYPTED WITH
(
    COLUMN_ENCRYPTION_KEY = CEK_GradosTitulos,
    ENCRYPTION_TYPE = DETERMINISTIC,
    ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
);
GO

-- Cifrar la columna Telefono
ALTER TABLE Academico.Persona
ALTER COLUMN Telefono VARCHAR(20)
COLLATE Latin1_General_BIN2
ENCRYPTED WITH
(
    COLUMN_ENCRYPTION_KEY = CEK_GradosTitulos,
    ENCRYPTION_TYPE = RANDOMIZED,
    ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
);
GO

-- Verificar las columnas cifradas
SELECT
    name AS Columna,
    encryption_type_desc AS TipoCifrado
FROM sys.columns
WHERE object_id = OBJECT_ID('Academico.Persona');
GO

```

Justificación técnica de la solución aplicada

Se implementó **Always Encrypted** para proteger la información personal de los estudiantes, garantizando que los datos sensibles permanezcan cifrados tanto en almacenamiento como durante su transmisión hacia las aplicaciones cliente.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron claves independientes (**Column Master Key** y **Column Encryption Key**) y algoritmos de cifrado recomendados por SQL Server. Además, se verificó la configuración de las columnas para asegurar que la información confidencial quede protegida correctamente.

Proyecto 3: Cifrado de información de jurados y asesores

Enunciado

Proteja la información personal de asesores y jurados de tesis mediante **Always Encrypted**, garantizando la confidencialidad de sus datos personales almacenados en la base de datos **GradosTitulos**.

Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- Crear una Clave Maestra de Columna
CREATE COLUMN MASTER KEY CMK_Jurados
WITH
(
    KEY_STORE_PROVIDER_NAME = N'MSSQL_CERTIFICATE_STORE',
    KEY_PATH = N'CurrentUser/My/JuradosCMK'
);
GO

-- Crear una Clave de Cifrado de Columna
CREATE COLUMN ENCRYPTION KEY CEK_Jurados
WITH VALUES
(
    COLUMN_MASTER_KEY = CMK_Jurados,
    ALGORITHM = 'RSA_OAEP',
    ENCRYPTED_VALUE = 0x01
);
GO

-- Cifrar la columna DNI del asesor
ALTER TABLE Academico.Asesor
ALTER COLUMN DNI VARCHAR(8)
COLLATE Latin1_General_BIN2
ENCRYPTED WITH
(
    COLUMN_ENCRYPTION_KEY = CEK_Jurados,
    ENCRYPTION_TYPE = DETERMINISTIC,
    ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
);
GO

-- Cifrar la columna Correo del asesor
ALTER TABLE Academico.Asesor
ALTER COLUMN Correo VARCHAR(100)
COLLATE Latin1_General_BIN2
ENCRYPTED WITH
(
    COLUMN_ENCRYPTION_KEY = CEK_Jurados,
    ENCRYPTION_TYPE = DETERMINISTIC,
    ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
);
GO

-- Cifrar la columna DNI del jurado
ALTER TABLE Academico.Jurado
ALTER COLUMN DNI VARCHAR(8)
COLLATE Latin1_General_BIN2
ENCRYPTED WITH
(
    COLUMN_ENCRYPTION_KEY = CEK_Jurados,
    ENCRYPTION_TYPE = DETERMINISTIC,
    ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
);
GO

-- Cifrar la columna Correo del jurado
ALTER TABLE Academico.Jurado
ALTER COLUMN Correo VARCHAR(100)
COLLATE Latin1_General_BIN2
ENCRYPTED WITH
(
    COLUMN_ENCRYPTION_KEY = CEK_Jurados,
    ENCRYPTION_TYPE = DETERMINISTIC,
    ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
);
GO

-- Verificar las columnas cifradas
SELECT
    name AS Columna,
    encryption_type_desc AS TipoCifrado
FROM sys.columns
WHERE object_id IN
(
    OBJECT_ID('Academico.Asesor'),
    OBJECT_ID('Academico.Jurado')
);
GO

```

Justificación técnica de la solución aplicada

Se implementó Always Encrypted para proteger los datos personales de asesores y jurados, garantizando que la información permanezca cifrada y solo pueda ser visualizada por aplicaciones autorizadas con acceso a las claves de cifrado.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron Column Master Key y Column Encryption Key independientes, junto con algoritmos de cifrado recomendados por SQL Server. Además, se verificó la configuración de las columnas para asegurar que la información confidencial se encuentre protegida.

TEMA 6. Auditoría y monitoreo de eventos con SQL Server Audit

Proyecto 1: Auditoría de accesos a la base de datos

Enunciado

Registrar todos los accesos realizados a la base de datos **GradosTitulos**.

Script de la solución en T-SQL

```

USE master;
GO

-- Crear la auditoría del servidor
CREATE SERVER AUDIT AuditoriaGradosTitulos
TO FILE
(
    FILEPATH = 'D:\Auditoria\'
);
GO

-- Habilitar la auditoría
ALTER SERVER AUDIT AuditoriaGradosTitulos
WITH (STATE = ON);
GO

USE GradosTitulos;
GO

-- Crear la especificación de auditoría
CREATE DATABASE AUDIT SPECIFICATION AuditoriaAccesosBD
FOR SERVER AUDIT AuditoriaGradosTitulos
ADD (SUCCESSFUL_DATABASE_AUTHENTICATION_GROUP),
ADD (FAILED_DATABASE_AUTHENTICATION_GROUP);
GO

-- Habilitar la especificación
ALTER DATABASE AUDIT SPECIFICATION AuditoriaAccesosBD
WITH (STATE = ON);
GO

-- Verificar las auditorías configuradas
SELECT
    name AS Auditoria,
    is_state_enabled AS Estado
FROM sys.server_audits;
GO

```

Justificación técnica de la solución aplicada

Se implementó **SQL Server Audit** para registrar los accesos exitosos y fallidos a la base de datos **GradosTitulos**, permitiendo llevar un control de las conexiones realizadas por los usuarios.

Explicación de las buenas prácticas utilizadas en el proyecto

Se almacenaron los eventos de auditoría en archivos dedicados y se habilitó únicamente la captura de eventos de autenticación, facilitando el monitoreo y el análisis de accesos al sistema.

Proyecto 2: Auditoría de modificaciones en trámites académicos

Enunciado

Registrar todas las operaciones **INSERT**, **UPDATE** y **DELETE** realizadas sobre las tablas del esquema **Tramite**.

Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- Crear la especificación de auditoría para el esquema Tramite
CREATE DATABASE AUDIT SPECIFICATION AuditoriaTramite
FOR SERVER AUDIT AuditoriaGradosTitulos
ADD (INSERT, UPDATE, DELETE
ON SCHEMA::Tramite
BY PUBLIC);
GO

-- Habilitar la especificación de auditoría
ALTER DATABASE AUDIT SPECIFICATION AuditoriaTramite
WITH (STATE = ON);
GO

-- Verificar las especificaciones de auditoría
SELECT
    name AS Auditoria,
    is_state_enabled AS Estado
FROM sys.database_audit_specifications;
GO

```

Justificación técnica de la solución aplicada

Se configuró una especificación de auditoría para registrar todas las operaciones de inserción, actualización y eliminación realizadas sobre el esquema **Tramite**, permitiendo mantener un historial de los cambios efectuados.

Explicación de las buenas prácticas utilizadas en el proyecto

Se centralizó la auditoría utilizando **SQL Server Audit** sobre todo el esquema **Tramite**, facilitando el seguimiento de las operaciones críticas y fortaleciendo el control sobre la información administrativa.

Proyecto 3: Sistema integral de auditoría institucional

Enunciado

Diseñar una solución de auditoría que registre:

- Inicio de sesión.
- Cambios de permisos.
- Creación de usuarios.
- Eliminación de objetos.
- Modificaciones de datos críticos.

Script de la solución en T-SQL

```
USE master;
GO

-- Crear la especificación de auditoría del servidor
CREATE SERVER AUDIT SPECIFICATION AuditoriaInstitucional
FOR SERVER AUDIT AuditoriaGradosTitulos
ADD (SUCCESSFUL_LOGIN_GROUP),
ADD (FAILED_LOGIN_GROUP),
ADD (SERVER_PERMISSION_CHANGE_GROUP),
ADD (SERVER_PRINCIPAL_CHANGE_GROUP);
GO

-- Habilitar la auditoría del servidor
ALTER SERVER AUDIT SPECIFICATION AuditoriaInstitucional
WITH (STATE = ON);
GO

USE GradosTitulos;
GO

-- Crear la especificación de auditoría de la base de datos
CREATE DATABASE AUDIT SPECIFICATION AuditoriaBD
FOR SERVER AUDIT AuditoriaGradosTitulos
ADD (SCHEMA_OBJECT_CHANGE_GROUP),
ADD (DATABASE_OBJECT_PERMISSION_CHANGE_GROUP),
ADD (INSERT, UPDATE, DELETE
ON SCHEMA::Academico
BY PUBLIC),
ADD (INSERT, UPDATE, DELETE
ON SCHEMA::Tramite
BY PUBLIC);
GO

-- Habilitar la auditoría de la base de datos
ALTER DATABASE AUDIT SPECIFICATION AuditoriaBD
WITH (STATE = ON);
GO

-- Verificar las auditorías configuradas
SELECT
    name AS Auditoria,
    is_state_enabled AS Estado
FROM sys.database_audit_specifications;
GO
```

Justificación técnica de la solución aplicada

Se implementó un sistema integral de auditoría mediante SQL Server Audit, registrando eventos de inicio de sesión, cambios de permisos, creación de usuarios, modificaciones de objetos y operaciones sobre datos críticos de la base de datos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se centralizó la auditoría utilizando una única configuración de SQL Server Audit, diferenciando los eventos del servidor y de la base de datos para facilitar el monitoreo, la trazabilidad y el cumplimiento de las políticas de seguridad institucional.